

Laboratory Work No.1

“Regular Grammars”

Course: Formal Languages & Finite Automata

Name: Turcanu Vladislav FAF-232

Contents

- **Objectives**
 - Implementation
 - Formal Languages
 - Project Setup
 - Grammars
 - Finite Automata
 - Results
- **Conclusion**

Objectives

Understanding Formal Languages:

Discover what a language is and what it needs to have in order to be considered a formal one.

Project Setup:

Provide the initial setup for the evolving project to be worked on during this semester.

Create a GitHub repository for storing and updating the project.

Choose a programming language, prioritizing simplicity in problem-solving.

Store reports separately to simplify verification of work.

Grammar Implementation:

Implement a type/class for the grammar.

Add a function to generate 5 valid strings from the language expressed by the grammar.

Finite Automaton Conversion:

Implement functionality to convert an object of type Grammar to one of type Finite Automaton.

Add a method to the Finite Automaton class to check if an input string can be obtained via state transition from it.

Implementation

Characteristics of Formal Languages

There is a lot we can speak about about the topic of Formal Languages. These are some of our symbolistics that we care a lot about while speaking about Formal Languages.

Alphabet - is a set of symbols on which this language is built.

Strings - is a set of strings of symbols drawn from a finite alphabet.

Syntax - are precise rules that tell you the symbols you are allowed to use and how to put them together into legal expressions.

Semantics - are precise rules that tell you the meanings of the symbols and legal expressions.

Formal Grammar - is defined as a set of production rules for such strings in a formal language.

Automata Theory - is a set of strings of symbols drawn from a finite alphabet.

Grammars

To implement our grammar I created a file with a class Grammar. To actually visualize some of the strings that can be created out of our grammar, I made a new Method in our main file to Generate Strings. This method will create any number of strings that we give it out of our grammar.

Grammar Class Implementation:

```
13 class Grammar: 2 usages  ⚙️ Vladis07
14     def __init__(self):  ⚙️ Vladis07
15         self.VN = {'S', 'A', 'B'}
16         self.VT = {'a', 'b', 'c'}
17         self.P = {
18             'S': ['aA', 'bB'],
19             'B': ['aB', 'b'],
20             'A': ['cA', 'aB', 'bS'],
21         }
```

In this code we have a class named Grammar that has everything we need for our grammar implementation with the variant we have (Variant 27).

```

7      def generate_valid(grammar, nstrings): 1 usage  Vladis07
8          valid = []
9
10         def generate_string(remaining, transition): Vladis07
11             if not remaining:
12                 return '', transition
13             for index, char in enumerate(remaining):
14                 if char.isupper():
15                     current = char
16                     break
17             else:
18                 return remaining, transition
19
20             if current in grammar.VT:
21                 current_str, transition = generate_string(remaining[index + 1:], transition)
22                 return current + current_str, transition
23             else:
24                 product = random.choice(grammar.P[current])
25                 new_remaining = ''.join(reversed(product)) + remaining[index + 1:]
26                 transition.append((current, product))
27                 return generate_string(new_remaining, transition)
28
29         for _ in range(nstrings):
30             string, transitions = generate_string(remaining: 'S', transition: [('S', 'S')])
31             valid.append((string[::-1], transitions))
32
33         return valid

```

In this code we have our function to take the grammar and a number of strings as input and generate the specified number of final strings that are valid for our grammar through random tries. Each string needs to transit through some transitions that are used to derive a valid string.

Finite Automata

For our Finite Automata Implementation, we need a class that I named Finite_Automata. In this class we created methods to convert grammar to a finite automata and it will check if the input string can be obtained via our transitions.

```

class Finite_Automata: 2 usages  👤 Vladis07
    def __init__(self, grammar):  👤 Vladis07
        self.alphabet = {}
        self.state = {}
        self.transition = {}
        self.init_state = None
        self.final_state = {}

        self.convert_grammar(grammar)

```

```

11         def convert_grammar(self, grammar): 1 usage  👤 Vladis07
12             self.alphabet = grammar.VT
13             self.state = grammar.VN
14             for symb in grammar.P:
15                 for product in grammar.P[symb]:
16                     if len(product) == 1:
17                         self.transition[(symb, product)] = 'final'
18                     else:
19                         self.transition[(symb, product[0])] = product[1]
20
21             self.init_state = 'S'
22             self.final_state = {symb for symb in grammar.P if symb.isupper()}
23

```

This will take a grammar object and will convert it to its finite automata. It sets the basics to create finite automata with the grammar object we provided.

Checking the input string

```

24  ✓    def check(self, input_string): 1 usage  👤 Vladis07
25        current_state = self.init_state
26  ✓    for char in input_string:
27  ✓        if not (current_state, char) in self.transition:
28            return False
29        current_state = self.transition[(current_state, char)]
30
31        return True

```

This method will take some input string and it will check if it can be obtained with our transitions from the finite automata. It will iterate through every character then it will update the current state via the transitions defined. If it will reach the final

state it will return True, if the current state was not in the transition it will return False.

Results:

```
Strings generated:
-> S -> bB -> b -> bb

-> S -> bB -> b -> bb

-> S -> aA -> bS -> bB -> b -> abbb

-> S -> aA -> aB -> aB -> b -> aaab

-> S -> aA -> aB -> b -> aab

Checking if the strings can be created
'abac' can be created
'baba' cannot be created
'wwa' cannot be created
'cca' cannot be created
'aaaaac' cannot be created
```

Conclusions:

In this lab we built the basics of Finite Automata. We made our own repository to setup our project for today and the future. We organised the work flow and reports building. We learned how to convert grammar to Finite Automata and how to generate strings. How to check if they can be created or not.